

Group assignment 1

Ben Bünnagel, Finn Hetzler, Nils Rudolf, Rena Schaller

MSc. Business Analytics

2615: Web and Cloud Computing

Nova School of Business and Economics

February 2025

1 Introduction

Job search is increasingly important, especially for soon to be Masters graduates approaching their first full-time employment. But often, tracking the availability of different job offers is a time-consuming and tedious task, managing different job platforms, with job offers sometimes appearing on multiple platforms, sometimes just on a single one.

Therefore, maintaining an overview of job offers is not always straightforward and requires constant monitoring. After receiving similar input on job search from our peers, we chose to propose a combined job search application for Nova students, showing relevant jobs for certain departments across several job platforms, with the option to search for specific job offers and providing further information about employers and salaries. In the scope of the course 2615 Web and Cloud Computing, we were given the task of developing a web application that uses data from public application programming interfaces (APIs) to provide job offer monitoring as a service to a user, and we want to use this project as a platform to provide a meaningful contribution to the Nova community.

In the following, we will lay out our project proposition, including the used data sources and APIs, as well as the proposed methodology and system architecture, while showing a first visual prototype and our implementation roadmap for the next stages of the project.

The aim of this project is to contribute to our Nova community of peers by simplifying the job search for Master's graduates in Business Analytics, making job monitoring easier, and providing valuable insights for job application through interactive visualizations.

2 Data source and API analysis

2.1 JobSpy Repository Overview

The main source we want to use to scrape multiple job platforms for relevant job listings is a Python library called JobSpy. JobSpy is open source and is freely available on GitHub [1]. It is able to scrape jobs from LinkedIn, Indeed, Glassdoor, Google and

ZipRecruiter and present results for search queries regarding jobs on these platforms in a CSV or Excel format.

By default, in the output file, the following scraped information for each job is shown: site from which job is scraped, job URL, direct URL of job, job title, company name, location, date the job was posted, job type, salary information, remote/not remote, job level, job function, job description, company information, plus a few more fields specific to some of the scraped websites.

When searching for a job with JobSpy, the following optional parameters can be set to define the search: site from which job is scraped, search term, location, distance, job type, proxies the scraper will round-robin through to avoid IP banning for excessive scraping, remote/not remote, max nr. of results for each site, nr. of hours since job was posted and some more specific filters for some of the scraped websites.

2.2 API and API details

JobSpy itself is not an API but it leverages the APIs of both Glassdoor [2] and Indeed [3] to get job data via GraphQL. Meanwhile, for Google Jobs and LinkedIn raw HTML is scraped. Since we plan on using the Python library itself, we do not connect to any APIs via JobSpy directly.

Alternatively, if possible, we can also incorporate other free public APIs aggregating jobs, such as the Arbeitnow API [4], which benefits from not needing an API key. An alternative could be Arbeitsamt API [5], however, it does require an API key which may cause trouble if many NOVA students access our web app all over that API key.

2.3 Scraping Challenges

The biggest potential issue we face is the site's protections in place against excessive scraping or using of their APIs. This may result in issues of IP addresses being banned, requiring rotating proxies to ensure data is always scrapable when needed. More information on our exact system architecture, including how we plan on mitigating this problem, can be seen in the following section.

3 Project methodology and system architecture

Our project aims to develop a job board specifically tailored for Business Analytics students, allowing them to search for aggregated job listings from various job portals based on their German city location. We have adopted a modular approach to create a MVP that can be enhanced in future iterations. In the following sections, we describe our methodology and outline the system architecture, focusing on data acquisition and storage, the server-side application, and the client-side interface, while also discussing potential future enhancements.

3.1 Data Acquisition and Storage

Our data collection strategy employs web scraping techniques implemented through Google Colab, utilizing an adapted version of JobSpy, originally developed for the American market. We focus on European job platforms including LinkedIn, Indeed, Glassdoor, and Google Jobs. We will scrape comprehensive job information including platform links, direct job URLs, descriptions, titles, company details, locations, posting dates, job types, remote work status, job levels, functions, industry classifications, and company logos.

We chose Google Colab for its execution environment, hypothesizing that its trusted server infrastructure would mitigate the need for proxy rotation typically required for large-scale web scraping. While Indeed’s endpoint proves most reliable with no rate limiting, LinkedIn’s more restrictive nature typically caps results around the tenth page per IP address. To maintain data relevance, we incorporate timestamp information for each scraping operation, enabling future cleaning of outdated listings.

For our MVP demonstration, we utilize Google Sheets as our database solution, leveraging its API capabilities for JSON data queries. This choice is based on its suitability for our demonstration purposes and data volume. For future scaling, we envision transitioning to a Raspberry Pi-based solution for both scraping operations and database management.

3.2 Server side implementation

Our backend implementation utilizes Flask and incorporates three main components:

1. **Landing Page** We developed a standardized HTML interface featuring a city-input form and job market analytics visualizations specifically designed for Business Analytics students. These analytics are cached locally in SQLite and updated continuously based on scraped data.
2. **Job Listing Processing** Upon form submission, the server processes the city input and queries the Google Sheets API to retrieve job listings based on the requested city.
3. **Data Management** The system maintains a local SQLite database for caching frequently accessed data and supporting features such as city name auto-completion, ensuring consistent syntax between user inputs and database queries.

3.3 Client-Side architecture

The frontend implementation employs HTML, CSS, and JavaScript, following the Nova design system. Key features include:

1. **Interactive Search Interface** The landing page features an intelligent search form with real-time city suggestion functionality, querying the local SQLite database as users type to ensure accurate matching between user inputs and database entries.
2. **Job Listing Visualization** We implemented a dual-panel layout similar to LinkedIn's interface, with job listings (including company information, logos, job titles, and locations) displayed on the left panel and detailed job descriptions with direct application links on the right panel.
3. **Data Visualization** The landing page incorporates interactive charts and maps to provide market insights, enhancing the user experience for our target audience of Business Analytics students.

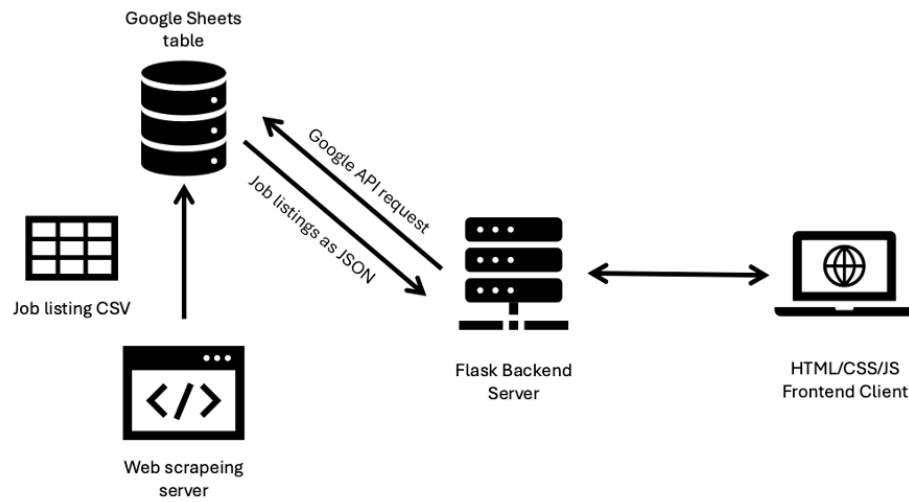


Figure 1: Visualization of the System Architecture

4 Prototype and Implementation Roadmap

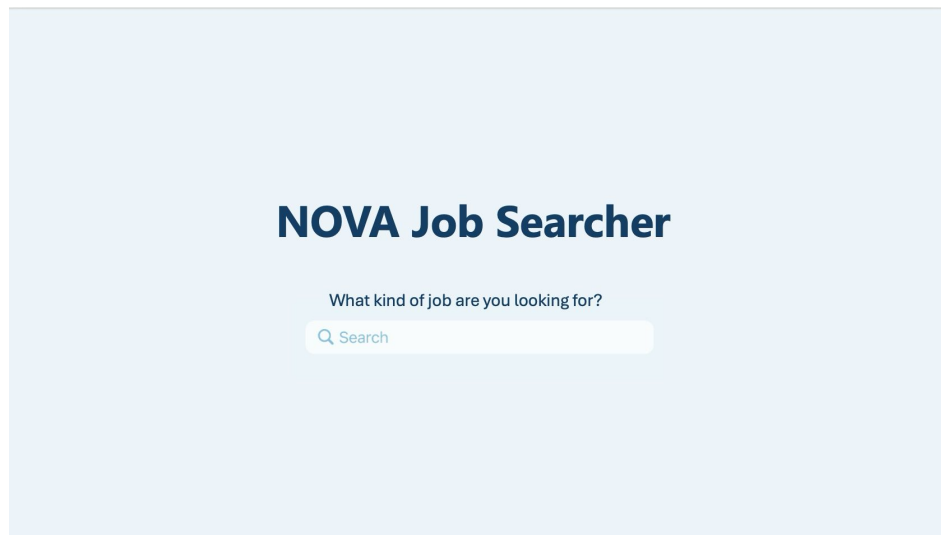


Figure 2: Landing page

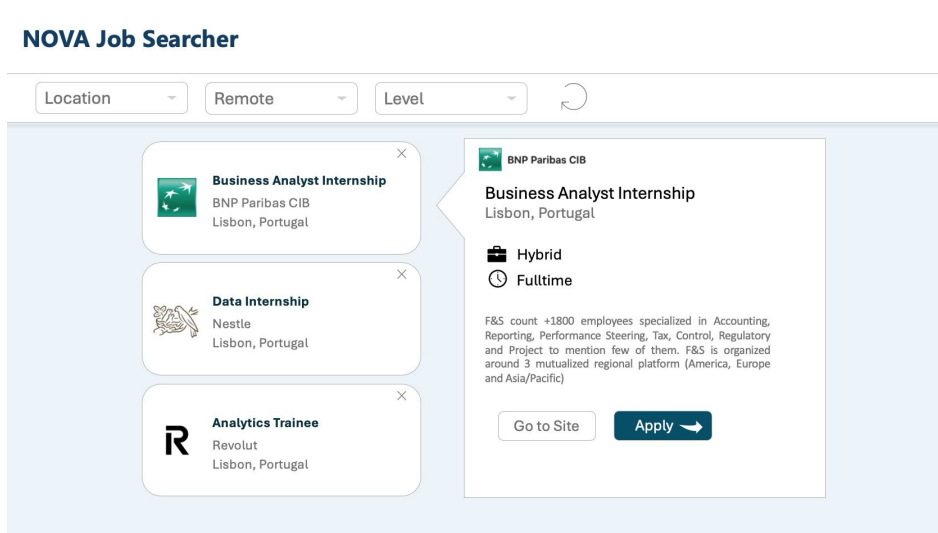


Figure 3: Search results UI

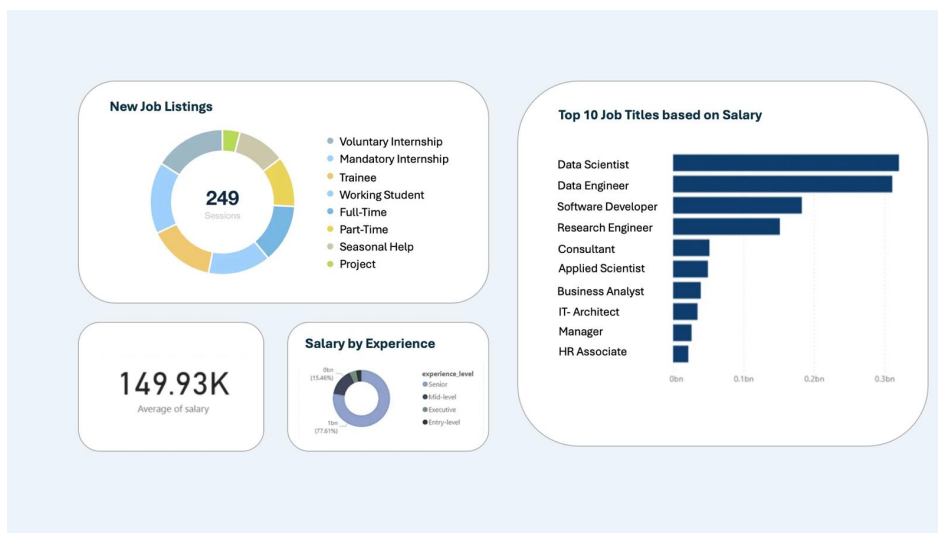


Figure 4: Analytics page

5 Conclusion

We propose to build a jobscraping service application, using Flask for the backend (server side application), that receives user queries from an HTML/CSS/JS frontend and then fetches these job queries from Google sheets using the Google sheets API and SQLite for caching frequently accessed data. The job listings are constantly updated via a web scraping server, so that the jobs are returned in JSON format to the flask backend. Rather than directly using available APIs, we therefore create our own API on top for aggregated and centralized job listings and additional information, offering a tailored service for Business Analytics students. Java script (JS) will be used to form a dynamic UI, enabling job loading from flask without requiring website reloading. Moreover, interactive job listings are displayed using Java Script, containing additional information about the job offer and showing detailed descriptions as well as application links. Furthermore, job market visualizations are presented, incorporating interactive charts for further job market insights. With this project, we aim to contribute to our academic community, implementing an innovative, unified API solution for Nova Business Analytics students, with a system that can expand beyond Nova, by transitioning to a more dedicated backend to incorporate even more jobs.

Bibliography

- [1] Github - cullenwatson/jobspy: Jobs scraper library for linkedin, indeed, glassdoor, google, bayt and ziprecruiter. Accessed February 23, 2025. URL: <https://github.com/cullenwatson/JobSpy>.
- [2] Glassdoor api documentation — glassdoor. Accessed February 23, 2025. URL: <https://www.glassdoor.com/developer/index.htm>.
- [3] Indeed api documentation. Accessed February 23, 2025. URL: <https://apis.indeed.com/>.
- [4] Arbeitnow. Accessed February 23, 2025. URL: <https://documenter.getpostman.com/view/18545278/UVJbJdKh>.
- [5] Arbeitsagentur jobsuche api - openapi documentation. Accessed February 23, 2025. URL: <https://jobsuche.api.bund.dev/>.